Seguridad del aplicativo: prueba por prueba

ESPE · Ingeniería de la Seguridad de Software

Vulnerabilidad. Defensa. Comando. Resultado. Captura.

20 casos concretos para ejecutar y explicar.
Cada comando repite únicamente la prueba indicada.

```
.venv/bin/python scripts/demo_case.py NOMBRE_DEL_CASO
```

Docker debe estar abierto. Las capturas se generaron de las ejecuciones reales guardadas.

Acceso a una cita ajena (BOLA)

R11 · P07

VULNERABILIDAD

Cambiar el identificador de una cita para consultar datos de otro paciente.

CÓMO SE DEFENDIÓ

El servidor verifica propietario, rol y ámbito en cada recurso.

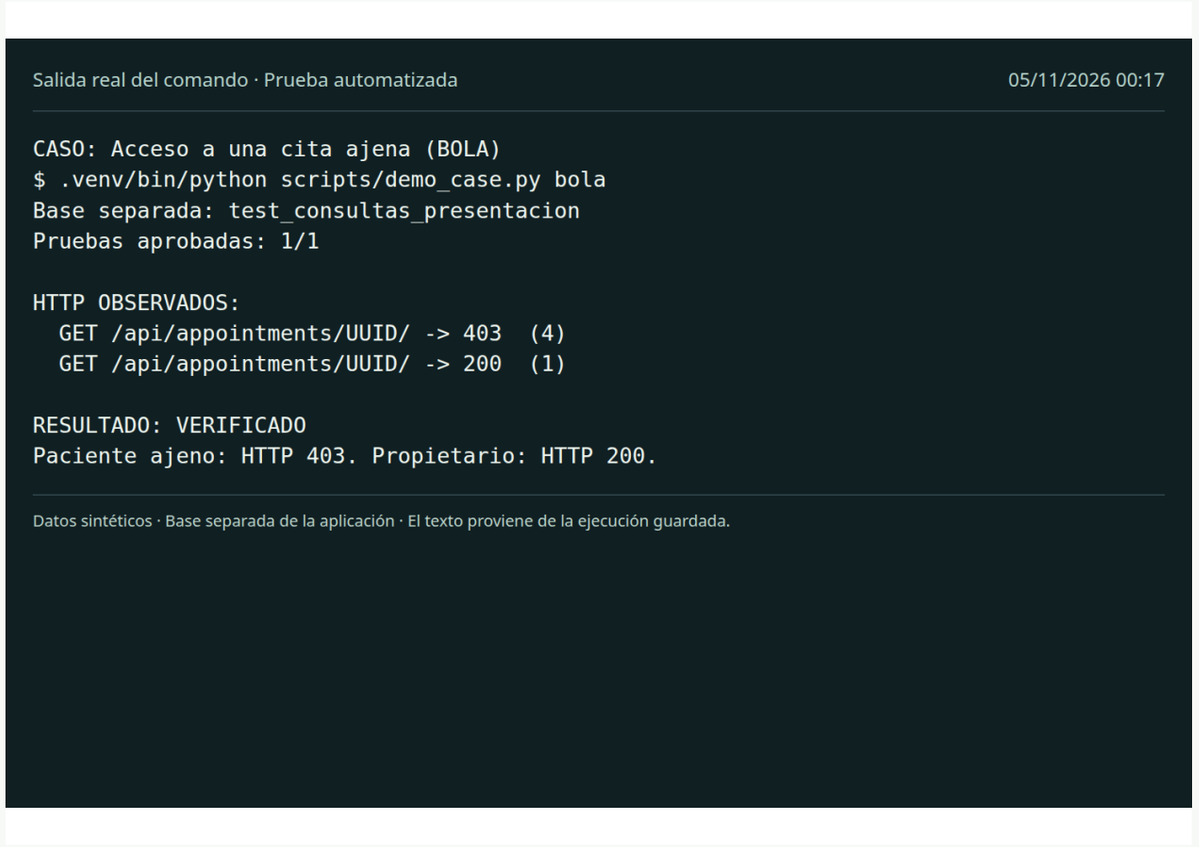
COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py bola
```

QUÉ HARÁ

Crea una cita de A y comprueba que B recibe 403, mientras A puede verla.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Paciente ajeno: HTTP 403. Propietario: HTTP 200.

Inyección SQL en el login

R15 · P13

VULNERABILIDAD

Introducir instrucciones SQL en un dato del formulario.

CÓMO SE DEFENDIÓ

Filtro de patrones, ORM parametrizado y usuario SQL sin permisos DDL.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py sql
```

QUÉ HARÁ

Envía la carga al login y prueba otra entrada que pasa el filtro sin alterar la consulta.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Payload bloqueado: 400. Entrada tratada como dato: sin resultados ajenos.

Inyección de JavaScript (XSS)

P13 · Defensa en profundidad

VULNERABILIDAD

Introducir JavaScript para que se ejecute en el navegador.

CÓMO SE DEFENDIÓ

Filtro de entrada, escape de HTML y política CSP de scripts.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py xss
```

QUÉ HARÁ

Envía un script al login y verifica el escape de contenido almacenado.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Login: HTTP 400. Contenido almacenado escapado y CSP presente.

Fuerza bruta de contraseñas

R13 · P09

VULNERABILIDAD

Probar contraseñas repetidamente hasta obtener acceso.

CÓMO SE DEFENDIÓ

Cuotas por cuenta e IP, error genérico, MFA y auditoría.

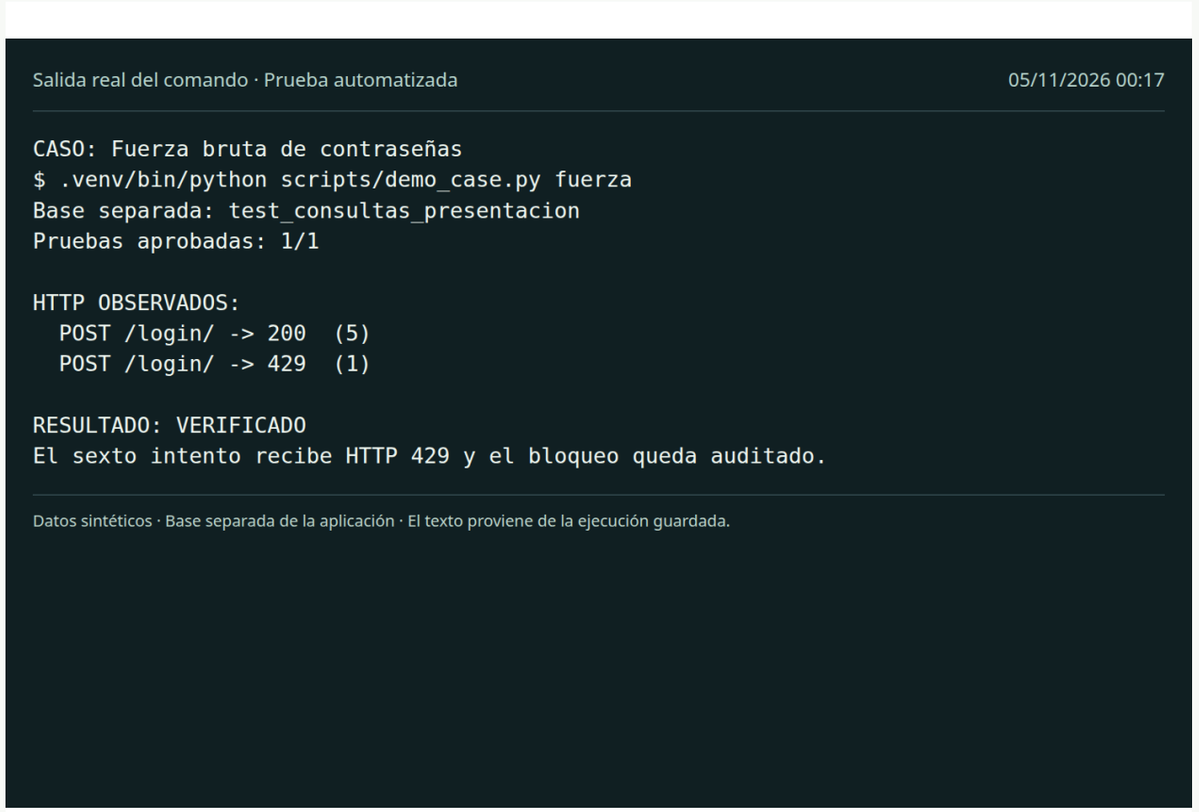
COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py fuerza
```

QUÉ HARÁ

Hace seis intentos erróneos con un usuario ficticio en una base separada.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



El sexto intento recibe HTTP 429 y el bloqueo queda auditado.

Saturación de solicitudes (DoS)

R14 · Cuota de aplicación

VULNERABILIDAD

Enviar demasiadas solicitudes para agotar recursos de la API.

CÓMO SE DEFENDIÓ

Límite de 300 solicitudes por IP y minuto; respuesta 429.

COMANDO A EJECUTAR

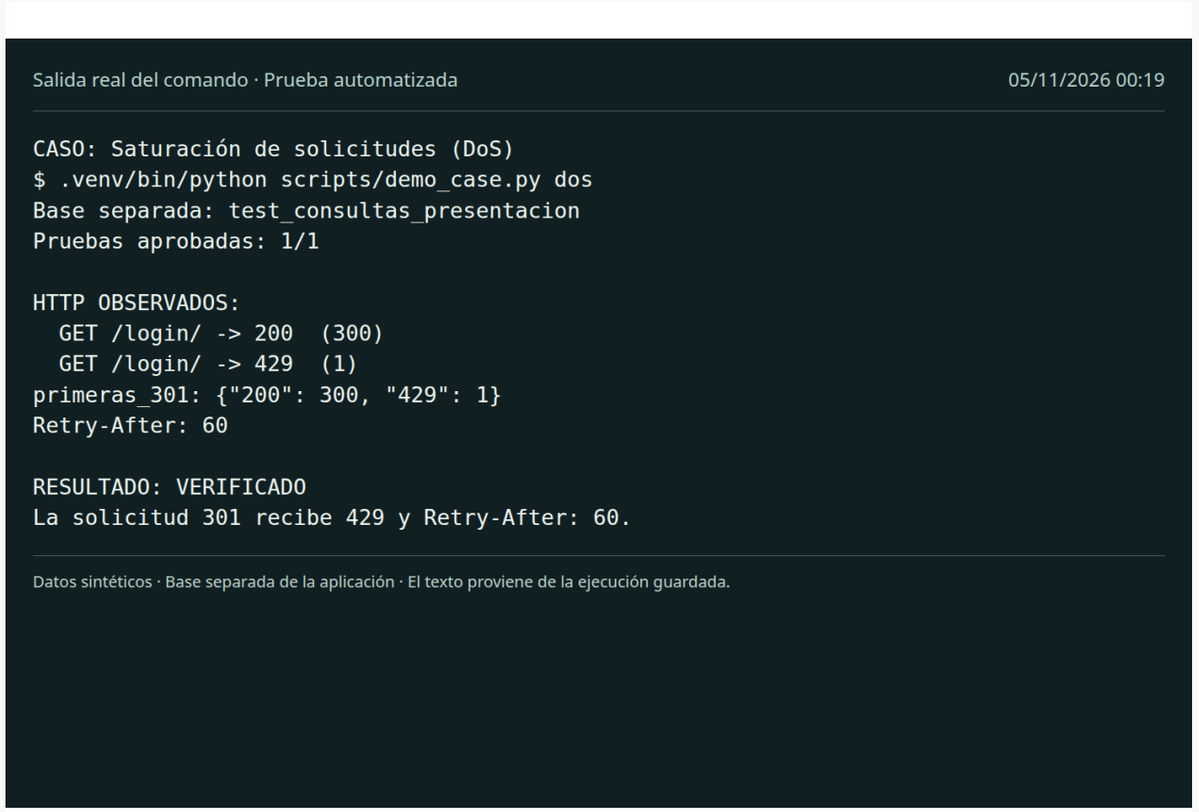
```
.venv/bin/python \
scripts/demo_case.py dos
```

QUÉ HARÁ

Hace 301 solicitudes aisladas: comprueba 300 respuestas 200 y una 429.

Prueba de cuota con cliente Django; no mide resistencia a un DDoS de red.

CAPTURA DEL RESULTADO · 05/11/2026 00:19



La solicitud 301 recibe 429 y Retry-After: 60.

Reutilizar un código MFA anterior

R04, R13 · P04, P09

VULNERABILIDAD

Intentar usar un código consumido o de otro inicio de sesión.

CÓMO SE DEFENDIÓ

Desafío por sesión previa, un uso, caducidad y nuevo código en cada login.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py mfa
```

QUÉ HARÁ

Entra, sale, crea otro acceso y compara el rechazo del código viejo con el nuevo.

La entrega por consola simula el canal del OTP de las cuentas sintéticas.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



El código anterior no abre el panel; el código nuevo sí.

Confirmación de pago falsificada

R16 · P06

VULNERABILIDAD

Marcar una orden como pagada con un webhook de firma falsa.

CÓMO SE DEFENDIÓ

Validación HMAC del emisor antes de procesar el pago.

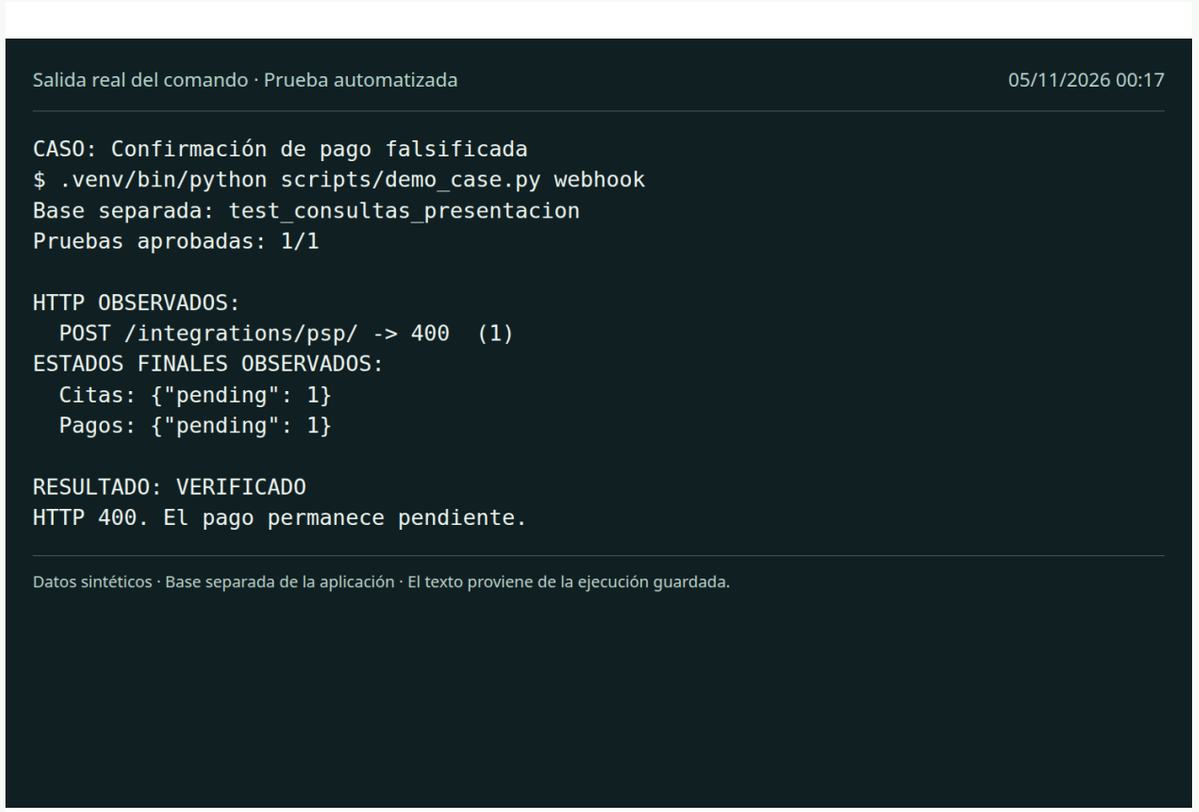
COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py webhook
```

QUÉ HARÁ

Crea una orden pendiente y envía su confirmación con una firma inválida.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



HTTP 400. El pago permanece pendiente.

Manipular monto o moneda

R16 · P06

VULNERABILIDAD

Cambiar el monto o la moneda de una orden para pagar menos.

CÓMO SE DEFENDIÓ

Comparación contra la orden real, aunque el mensaje tenga firma válida.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py monto
```

QUÉ HARÁ

Envía un evento firmado con un centavo y moneda EUR sobre una orden USD.

CAPTURA DEL RESULTADO · 05/11/2026 00:17

Salida real del comando · Prueba automatizada — 05/11/2026 00:17

```
CASO: Manipular monto o moneda
$ .venv/bin/python scripts/demo_case.py monto
Base separada: test_consultas_presentacion
Pruebas aprobadas: 1/1

HTTP OBSERVADOS:
  POST /integrations/psp/ -> 400  (1)
ESTADOS FINALES OBSERVADOS:
  Citas: {"pending": 1}
  Pagos: {"pending": 1}

RESULTADO: VERIFICADO
HTTP 400. La cita no pasa a confirmada.
```

Datos sintéticos · Base separada de la aplicación · El texto proviene de la ejecución guardada.

HTTP 400. La cita no pasa a confirmada.

Repetir un mensaje de integración

R10 · P05

VULNERABILIDAD

Reenviar un mensaje firmado o utilizar uno vencido.

CÓMO SE DEFENDIÓ

Nonce único por socio, timestamp y scope permitido.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py replay
```

QUÉ HARÁ

Repite un mensaje válido y ensaya una fecha antigua y un scope prohibido.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Primer envío: 200. Repetido/vencido: 400. Scope incorrecto: 403.

Duplicar cobros o reembolsos

S11 · P06, P08

VULNERABILIDAD

Repetir una operación para generar efectos financieros duplicados.

CÓMO SE DEFENDIÓ

Idempotencia, referencias únicas y transacciones de estado.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py cobros
```

QUÉ HARÁ

Repite un evento y una cancelación; verifica un solo cobro y reembolso.

CAPTURA DEL RESULTADO · 05/11/2026 00:17

Salida real del comando · Prueba automatizada — 05/11/2026 00:17

```
CASO: Duplicar cobros o reembolsos
$ .venv/bin/python scripts/demo_case.py cobros
Base separada: test_consultas_presentacion
Pruebas aprobadas: 2/2

HTTP OBSERVADOS:
  POST /integrations/psp/ -> 200  (2)
ESTADOS FINALES OBSERVADOS:
  Citas: {"confirmed": 1}
  Pagos: {"paid": 1}
  Reembolsos: 0
  Citas: {"cancelled": 1}
  Pagos: {"refunded": 1}
  Reembolsos: 1

RESULTADO: VERIFICADO
Un pago procesado y un único reembolso por orden.
```

Datos sintéticos · Base separada de la aplicación · El texto proviene de la ejecución guardada.

Un pago procesado y un único reembolso por orden.

Reservar el mismo cupo a la vez

R12 · P08

VULNERABILIDAD

Crear dos citas sobre el mismo horario mediante una carrera.

CÓMO SE DEFENDIÓ

Bloqueo de franja, transacción y restricción única parcial en PostgreSQL.

COMANDO A EJECUTAR

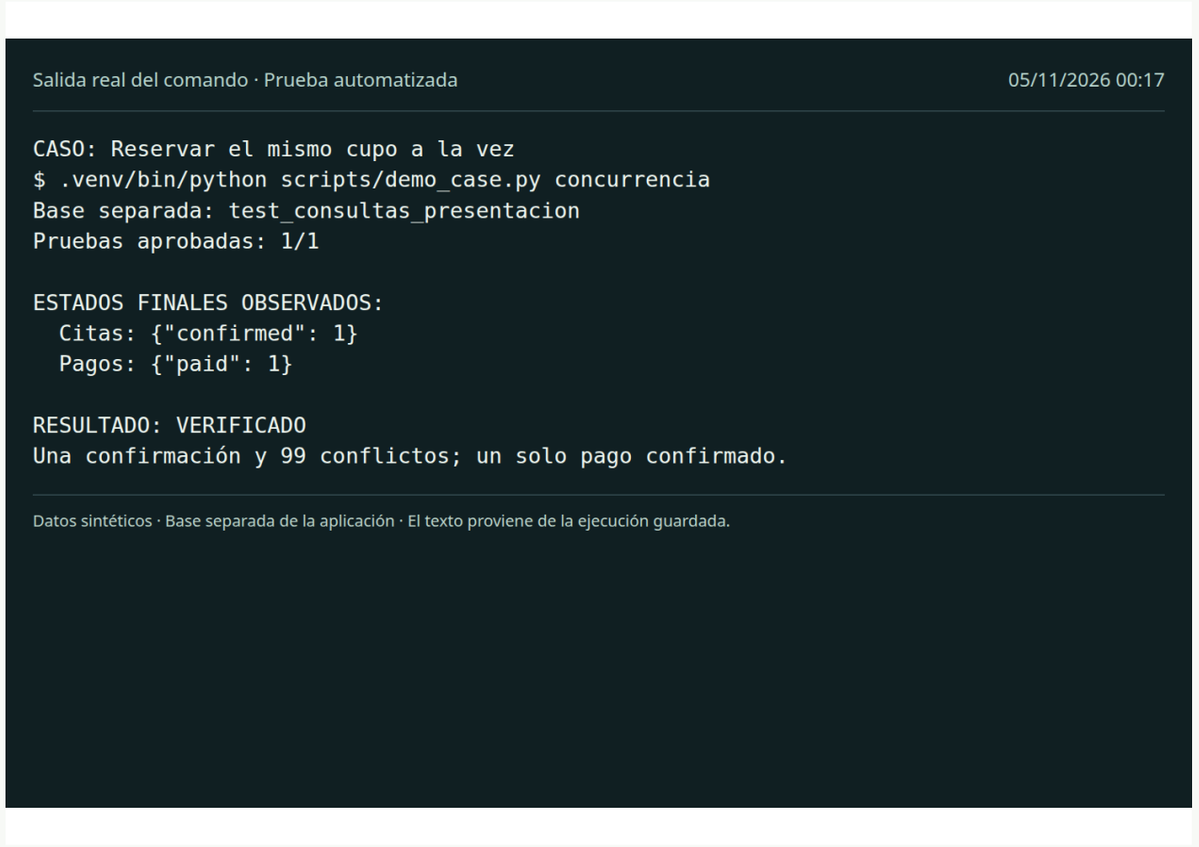
```
.venv/bin/python \
scripts/demo_case.py concurrencia
```

QUÉ HARÁ

Lanza 100 hilos con usuarios distintos sobre un mismo turno.

Concurrencia real de servicios y PostgreSQL; no son 100 clics manuales.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Una confirmación y 99 conflictos; un solo pago confirmado.

Cruzar centros, médicos o seguros

R02, R17 · P02, P05, P07

VULNERABILIDAD

Un actor interno intenta consultar información fuera de su ámbito.

CÓMO SE DEFENDIÓ

Filtros por organización, centro, profesional, póliza y beneficiario.

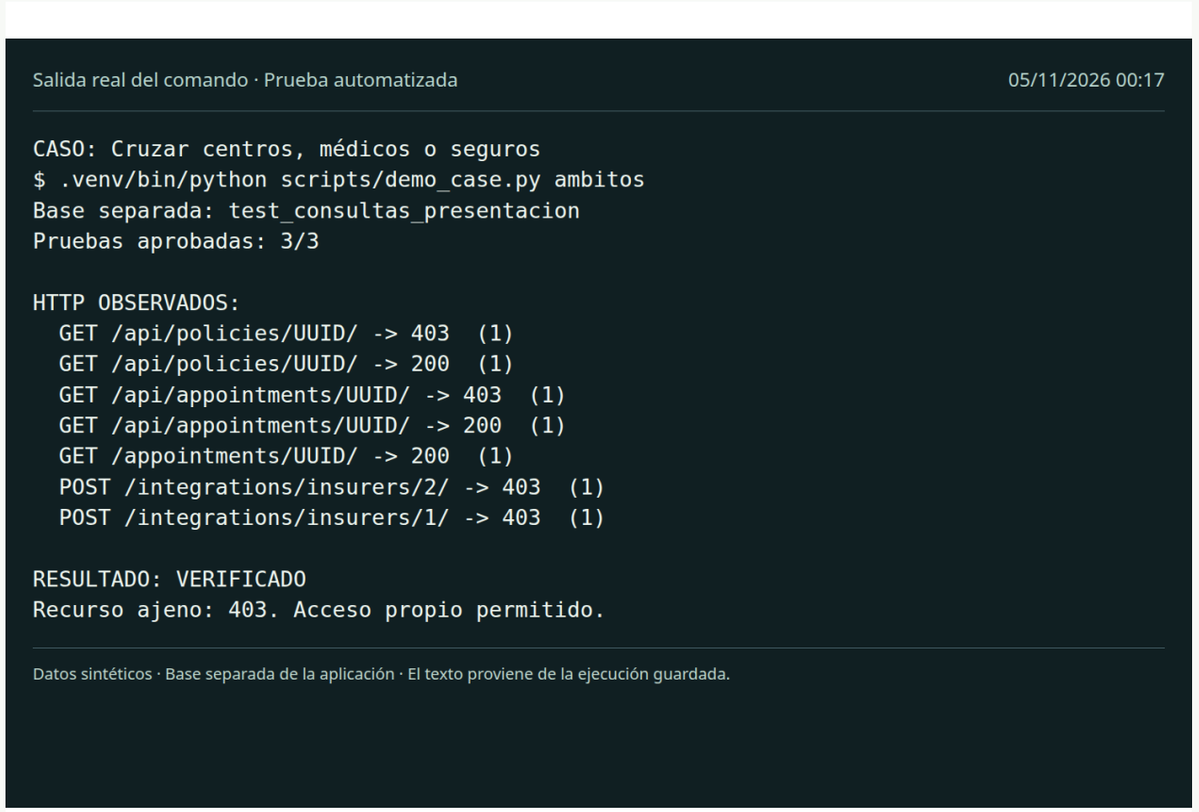
COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py ambitos
```

QUÉ HARÁ

Prueba una póliza de otro seguro y una cita de otro profesional.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Recurso ajeno: 403. Acceso propio permitido.

Escalar privilegios

R20 · Separación de funciones

VULNERABILIDAD

Recepción intenta usar funciones de operador o autoaprobar un rol.

CÓMO SE DEFENDIÓ

Permiso por función, dos operadores distintos y revocación de sesiones.

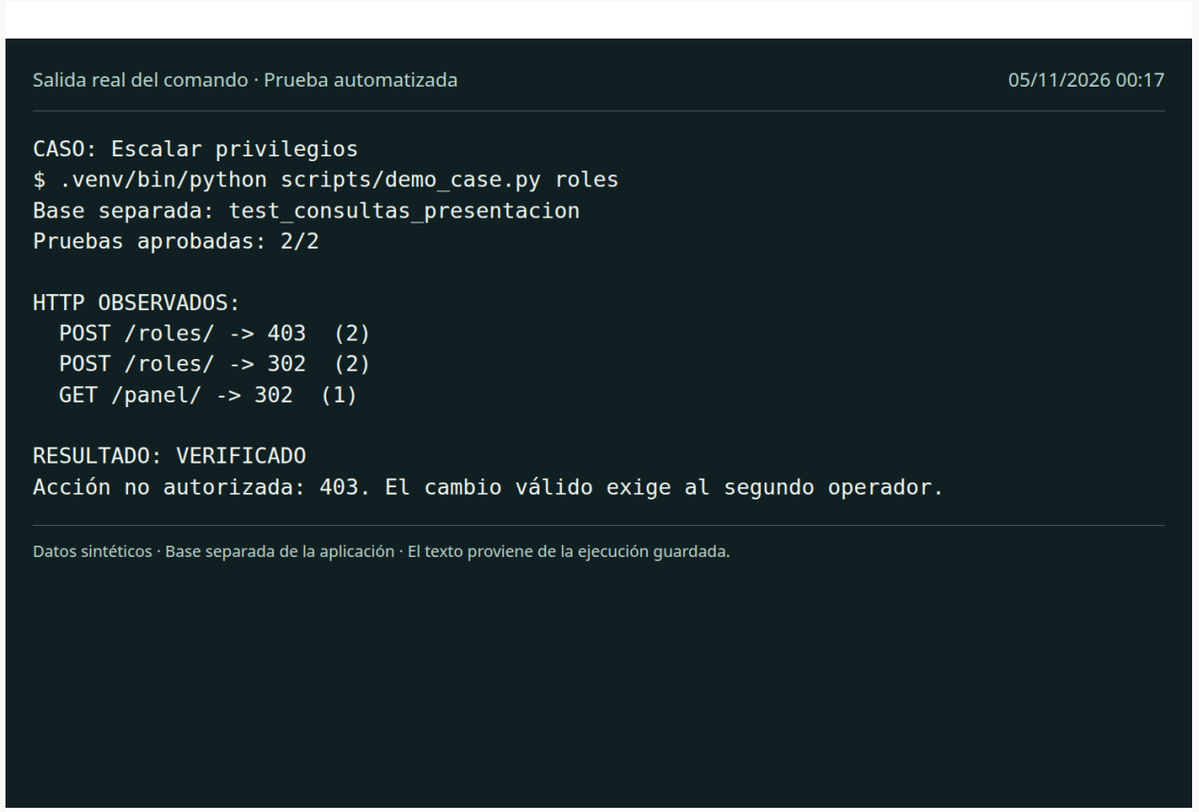
COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py roles
```

QUÉ HARÁ

Intenta elevar permisos y aprobar el propio pedido; después usa otro operador.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Acción no autorizada: 403. El cambio válido exige al segundo operador.

Representación falsa de un menor

R03 · P03

VULNERABILIDAD

Reservar para un dependiente sin acreditar representación.

CÓMO SE DEFENDIÓ

Solicitud con evidencia y revisión independiente antes de habilitar reservas.

COMANDO A EJECUTAR

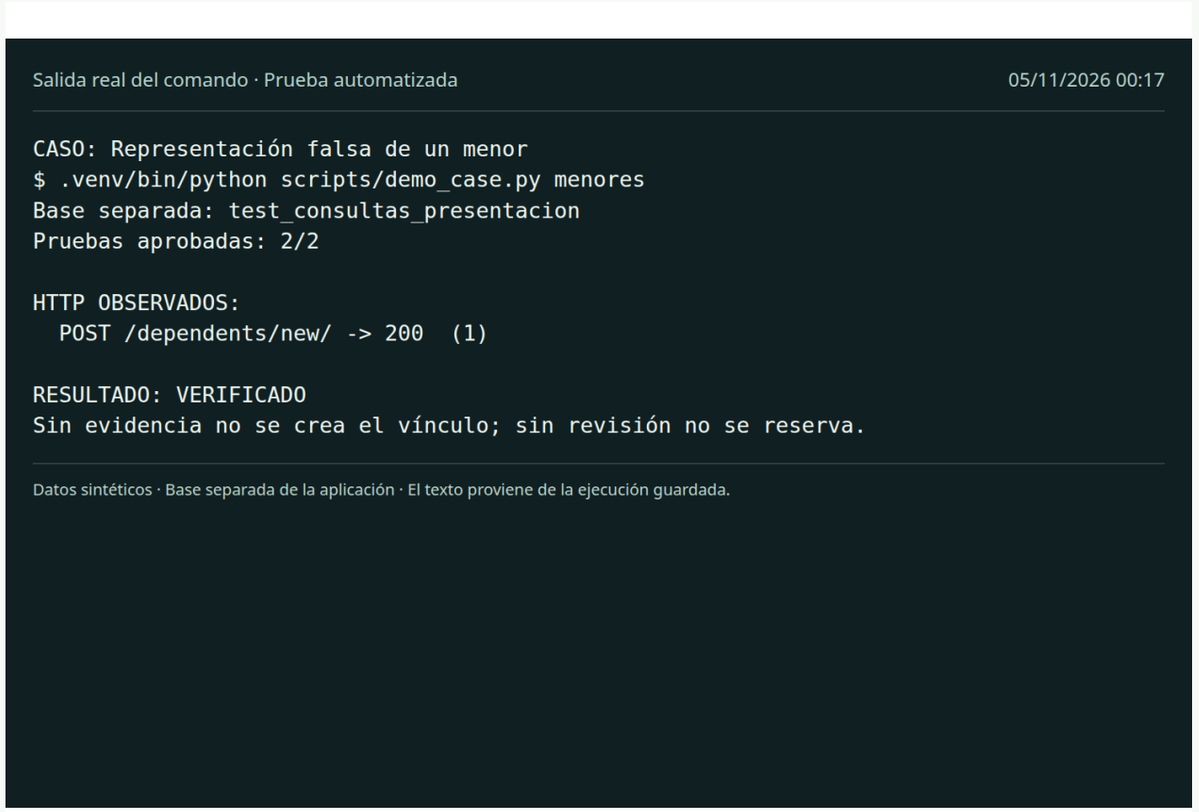
```
.venv/bin/python \
scripts/demo_case.py menores
```

QUÉ HARÁ

Prueba sin evidencia, sin revisión y después con vínculo validado.

La revisión usa evidencia ficticia; no acredita parentesco legal real.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Sin evidencia no se crea el vínculo; sin revisión no se reserva.

Reutilizar una sesión revocada

R04 · P04

VULNERABILIDAD

Volver a entrar con una cookie después de salir o tras su expiración.

CÓMO SE DEFENDIÓ

Sesión de servidor revocable y fecha de caducidad.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py sesion
```

QUÉ HARÁ

Reutiliza la cookie antigua y prueba otra sesión con fecha ya vencida.

Una cookie robada aún vigente conserva riesgo de uso.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



El panel devuelve 302 hacia login; no se recupera el acceso.

Alterar la auditoría

R06 · P11

VULNERABILIDAD

Modificar acciones registradas o dejar cambios sin responsable.

CÓMO SE DEFENDIÓ

Actores individuales, cadena HMAC y trigger append-only.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py auditoria
```

QUÉ HARÁ

Registra dos actores, verifica la cadena e intenta modificar un evento.

No demuestra resistencia a un administrador total del host y las claves.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Actores distintos, cadena íntegra y UPDATE rechazado.

Exponer claves y contraseñas

R18 · Revisión de archivos

VULNERABILIDAD

Publicar secretos reales en código o en archivos con permisos amplios.

CÓMO SE DEFENDIÓ

Secretos separados, archivos privados y exclusión de Git.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py secretos
```

QUÉ HARÁ

Revisa permisos y busca valores secretos en fuentes sin mostrarlos.

Escaneo local acotado; no sustituye Vault ni una auditoría de todo el host.

CAPTURA DEL RESULTADO · 05/11/2026 00:17

Salida real del comando · Prueba automatizada — 05/11/2026 00:17

```
CASO: Exponer claves y contraseñas
$ .venv/bin/python scripts/demo_case.py secretos
Base separada: test_consultas_presentacion
Pruebas aprobadas: 1/1

archivos_fuente_revisados: 63
coincidencias_secretos: 0
archivos_privados_verificados: 4

RESULTADO: VERIFICADO
Sin coincidencias de secretos reales en las fuentes revisadas.
```

Datos sintéticos · Base separada de la aplicación · El texto proviene de la ejecución guardada.

Sin coincidencias de secretos reales en las fuentes revisadas.

Exponer datos sensibles de más

R01, R05, R08, R09 · P01, P10

VULNERABILIDAD

Guardar clínica o tarjetas, o revelar datos médicos en un aviso.

CÓMO SE DEFENDIÓ

Minimización, póliza cifrada y plantilla neutral de notificación.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py privacidad
```

QUÉ HARÁ

Inspecciona el esquema, el cifrado de póliza y el texto de un aviso real.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



Sin campos clínicos/PAN/CVV y aviso sin diagnóstico ni póliza.

Conservar datos fuera de plazo

R19 · P14

VULNERABILIDAD

Mantener identificable una cuenta vencida o borrar datos con retención legal.

CÓMO SE DEFENDIÓ

Disociación por plazo, legal hold y registro de supresiones.

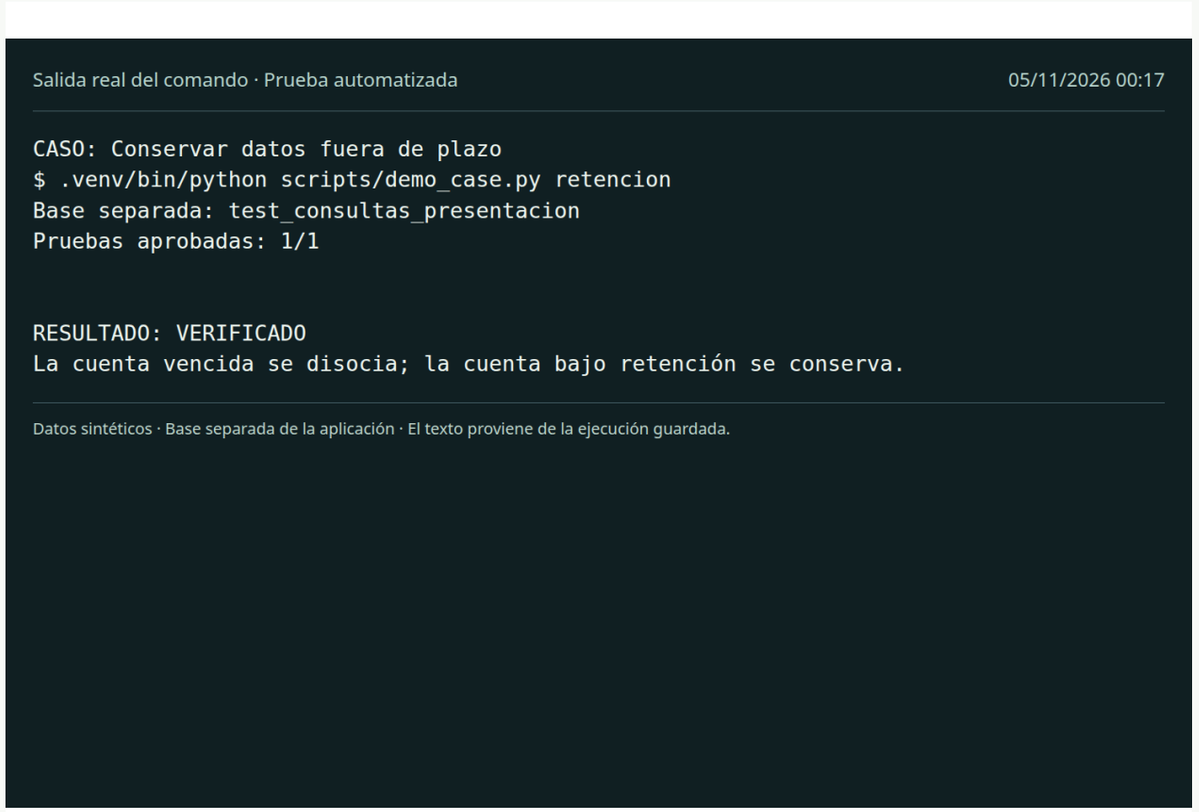
COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py retencion
```

QUÉ HARÁ

Vence dos cuentas ficticias; una tiene retención legal y la otra no.

CAPTURA DEL RESULTADO · 05/11/2026 00:17



La cuenta vencida se disocia; la cuenta bajo retención se conserva.

Pérdida o corrupción de la base

R07 · P12

VULNERABILIDAD

Perder las citas y no poder recuperar una copia utilizable.

CÓMO SE DEFENDIÓ

Copia cifrada, restauración en otra base y verificación de integridad.

COMANDO A EJECUTAR

```
.venv/bin/python \
scripts/demo_case.py backup
```

QUÉ HARÁ

Crea una cita sintética, hace backup y restaura en una base NUEVA de Docker.

Ensayo local; no simula pérdida del host ni demuestra alta disponibilidad.

CAPTURA DEL RESULTADO · 05/11/2026 00:17

Salida real del comando · Prueba automatizada — 05/11/2026 00:17

```
CASO: Pérdida o corrupción de la base
$ .venv/bin/python scripts/demo_case.py backup
Base separada: test_consultas_presentacion
Pruebas aprobadas: 1/1

checksum_verificado: True
auditoria_verificada: True
sesiones_revocadas: True
citas_restauradas: 1
pagos_restaurados: 1
tiempo_restauracion_s: 3.087
modo: Docker · base nueva

RESULTADO: VERIFICADO
Checksum y auditoría válidos; cita y pago restaurados; sesiones revocadas.
```

Datos sintéticos · Base separada de la aplicación · El texto proviene de la ejecución guardada.

Checksum y auditoría válidos; cita y pago restaurados; sesiones revocadas.